

3. Ejercicios con soluciones

Ejercicio 1

Usa la base de datos `mtcars`. Calcula, con un bucle `for`, la media de cada columna. Guarda los resultados en un vector numérico con nombres.

```
media_columna <- vector("double", ncol(mtcars))
name <- vector("double", ncol(mtcars))
for (columna in seq_along(mtcars)) {
  media_columna[[columna]] <- mean(mtcars[[columna]])
  name[[columna]] <- names(mtcars)[[columna]]
}
names(media_columna) <- c(name)
media_columna
```

```
##      mpg      cyl      disp      hp      drat      wt      qsec
## 20.090625  6.187500 230.721875 146.687500  3.596563  3.217250 17.848750
##      vs      am      gear      carb
##  0.437500  0.406250  3.687500  2.812500
```

Ejercicio 2

Usa la base de datos `iris`. Con un bucle `for`, determina qué columnas son numéricas. Guarda el resultado en un vector lógico con nombres.

```
column_numeric <- vector("logical", ncol(iris))
name <- vector("character", ncol(iris))

for (columna in seq_along(iris)) {
  column_numeric[[columna]] <- is.numeric(iris[[columna]])
  name[[columna]] <- names(iris)[[columna]]
}

names(column_numeric) <- name
column_numeric
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width Species
##      TRUE      TRUE      TRUE      TRUE      FALSE
```

Ejercicio 3

Usa la base de datos `iris`. Con un bucle `for`, calcula la desviación estándar únicamente de las columnas numéricas.

```
desviacion <- c()
name <- c()
for (i in seq_along(iris)){
  if (is.numeric(iris[[i]])){
    desviacion <- c(desviacion, sd(iris[[i]]))
  }
}
```

```

    name <- c(name,names(iris[i]) )
  }
}
names(desviacion) <- name
desviacion

```

```

## Sepal.Length Sepal.Width Petal.Length Petal.Width
##      0.8280661      0.4358663      1.7652982      0.7622377

```

Ejercicio 4

Crea una función llamada `resumen_columnas()` que reciba un data frame y una función. La función debe aplicar la función dada a cada columna del data frame.

Prueba tu función con `mean`, `median` y `sd` usando `mtcars`.

```

resumen_columnas <- function(dataframe,function){
  lista <- lapply(dataframe,function)
  unlist(lista)
}
resumen_columnas(mtcars,mean)

```

```

##      mpg      cyl      disp      hp      drat      wt      qsec
## 20.090625  6.187500 230.721875 146.687500  3.596563  3.217250 17.848750
##      vs      am      gear      carb
##  0.437500  0.406250  3.687500  2.812500

```

```
resumen_columnas(mtcars,median)
```

```

##      mpg      cyl      disp      hp      drat      wt      qsec      vs      am      gear
## 19.200    6.000 196.300 123.000    3.695    3.325 17.710    0.000    0.000    4.000
##      carb
##      2.000

```

```
resumen_columnas(mtcars,sd)
```

```

##      mpg      cyl      disp      hp      drat      wt
## 6.0269481  1.7859216 123.9386938 68.5628685  0.5346787  0.9784574
##      qsec      vs      am      gear      carb
## 1.7869432  0.5040161  0.4989909  0.7378041  1.6152000

```

Ejercicio 5

Adapta la función anterior para que trabaje únicamente con columnas numéricas. Pruébala con `iris`.

```

resumen_columnas_numericas <- function(dataframe,function){

  dataframe <- dataframe[, map_lgl(dataframe, is.numeric)]
  lista <- lapply(dataframe,function)
  unlist(lista)
}

```

```
}
```

```
resumen_columnas_numericas(iris,mean)
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width
##      5.843333      3.057333      3.758000      1.199333
```

```
resumen_columnas_numericas(iris,median)
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width
##           5.80           3.00           4.35           1.30
```

```
resumen_columnas_numericas(iris,sd)
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width
##      0.8280661      0.4358663      1.7652982      0.7622377
```

Ejercicio 6

Genera 10 números aleatorios normales para cada una de las siguientes medias: -10, 0, 10 y 100. Usa un bucle for y guarda los resultados en una lista.

```
medias <- c(-10, 0, 10, 100)
lista_medias <- list()
for (i in seq_along(medias)){
  lista_medias[[i]] <- rnorm(10,mean = medias[i])
}
names(lista_medias) <- paste("media",medias)
lista_medias
```

```
## $'media -10'
## [1] -10.694707 -10.207917 -11.265396 -7.831044 -8.792038 -11.123109
## [7] -10.402885 -10.466655 -9.220035 -10.083369
##
## $'media 0'
## [1] 0.25331851 -0.02854676 -0.04287046 1.36860228 -0.22577099 1.51647060
## [7] -1.54875280 0.58461375 0.12385424 0.21594157
##
## $'media 10'
## [1] 10.379639 9.497677 9.666793 8.981425 8.928209 10.303529 10.448210
## [8] 10.053004 10.922267 12.050085
##
## $'media 100'
## [1] 99.50897 97.69083 101.00574 99.29080 99.31199 101.02557 99.71523
## [8] 98.77928 100.18130 99.86111
```

Ejercicio 7

Reescribe el ejercicio anterior usando map().

```
medias <- c(-10, 0, 10, 100)
numeros <- function(numero){
  rnorm(10,mean=numero)
}

map(medias,numeros)
```

```
## [[1]]
## [1] -9.994236 -9.614720 -10.370660 -9.355623 -10.220487 -9.668218
## [7] -8.903161 -9.564819 -10.325932 -8.851192
##
## [[2]]
## [1] 0.9935039 0.5483970 0.2387317 -0.6279061 1.3606524 -0.6002596
## [7] 2.1873330 1.5326106 -0.2357004 -1.0264209
##
## [[3]]
## [1] 9.289593 10.256884 9.753308 9.652457 9.048381 9.954972 9.215096
## [8] 8.332058 9.619773 10.918997
##
## [[4]]
## [1] 99.42465 100.60796 98.38212 99.94444 100.51941 100.30115 100.10568
## [8] 99.35929 99.15030 98.97587
```

Ejercicio 8

Usa `map_dbl()` para calcular la media de cada columna de `mtcars`.

```
map_dbl(mtcars,mean)
```

```
##      mpg      cyl      disp      hp      drat      wt      qsec
## 20.090625  6.187500 230.721875 146.687500  3.596563  3.217250 17.848750
##      vs      am      gear      carb
##  0.437500  0.406250  3.687500  2.812500
```

Ejercicio 9

Usa `map_chr()` para obtener la clase de cada columna de `iris`.

```
map_chr(iris,class)
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width Species
##      "numeric"      "numeric"      "numeric"      "numeric"      "factor"
```

Ejercicio 10

Usa un bucle `while` para simular lanzamientos de una moneda hasta obtener dos caras seguidas. El resultado final debe ser el número de lanzamientos necesarios.

```
lanzamiento <- c("cara","corona")
contador <- 0
cara <- 0
while (cara<2) {
  resultado <- sample(lanzamiento,1)
  if (resultado == "cara"){
    cara <- cara+1
    contador <- contador+1
  }else{
    cara <- 0
    contador <- contador+1
  }
}
contador
```

```
## [1] 6
```